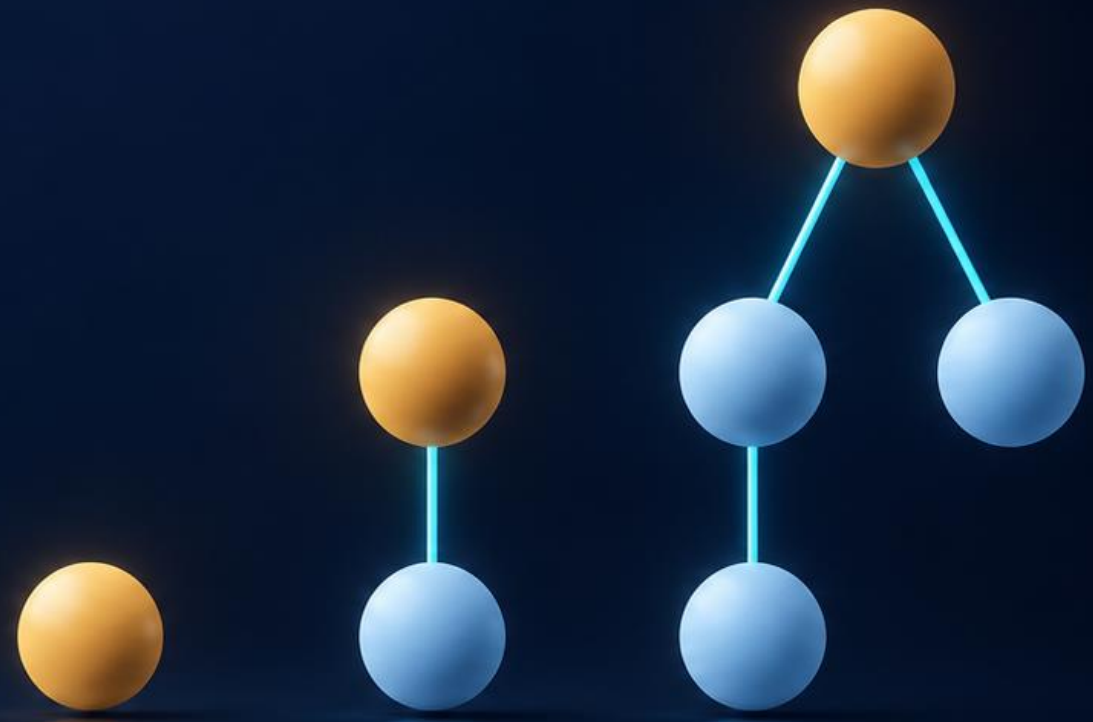
Биномиальные кучи

Устройство, слияние
и реализация на Python



Структуры данных

Очередь с приоритетом и слиянием

Биномиальная куча хранит элементы с приоритетами и поддерживает meld: объединение двух куч в одну.

Операции очереди

insert(x) добавить элемент

findMin() посмотреть минимум

extractMin() извлечь минимум

Объединение куч

meld(H_1, H_2) сохраняет элементы обеих очередей.

Биномиальная куча: O(log(n + m))

Для сравнения: объединение массивов бинарных куч с heapify занимает O(n + m).

Биномиальное дерево B□

Рекурсивное определение

B_0 состоит из одного узла. Два дерева B□ образуют B□$_{+1}$: корень одного становится ребёнком другого.

Свойства дерева порядка k

Число узлов	Высота	Детей у корня
2^k	k	k

На глубине i находится C(k, i) узлов. Отсюда название «биномиальное».

Биномиальная куча как лес

В min-heap ключ родителя ≤ ключей детей. Для каждого порядка k в куче есть не более одного дерева B□.

Набор деревьев задаёт двоичная запись размера кучи.

Порядок k	0	1	2	3
Бит числа 13	1	0	1	1
Дерево	B_0	—	B_2	B_3
Число узлов	1	0	4	8

13 = 1 + 4 + 8. Минимум находится среди корней этих трёх деревьев.

Слияние и переносы

Связывание двух деревьев

Объединяются деревья одного порядка.
Корень с меньшим ключом остаётся сверху, второй становится его ребёнком.

$B_{\square} + B_{\square} = B_{\square+1}$

Одна операция link требует O(1) изменений ссылок.

Пример: 5 + 3 = 8 элементов

$5 = B_2 + B_0$
$3 = B_1 + B_0$

$B_0 + B_0 = B_1$
$B_1 + B_1 = B_2$
$B_2 + B_2 = B_3$

Результат: одно дерево B_3.

Операции и сложность

Операция	Худший случай	Механизм
findMin	O(log n)	Просмотр корней
insert	O(log n)	Добавление B_0 и переносы
meld	O(log(n + m))	Связывание равных порядков
extractMin	O(log n)	Удаление корня и слияние детей
decreaseKey	O(log n)	Подъём ключа к корню

Вставка через переносы имеет O(1) амортизированную стоимость.

decreaseKey требует ссылки на узел. Поиск узла по ключу может занять O(n).

Способы хранения

Связные узлы

Узел хранит ключ и ссылки на родителя, первого ребёнка и следующего брата.

key, degree, parent
child, sibling

Корни образуют список, упорядоченный по степени дерева.

Коллекции Python

Детей узла удобно хранить в list. Корни можно хранить по порядку дерева.

roots[k] = корень B_k
children = [B_0, B_1, ...]

roots: list с пустыми слотами
или dict с ключами-порядками.

Python: узел и связывание

```
from dataclasses import dataclass, field

@dataclass
class Node:
    key: int
    children: list["Node"] = field(default_factory=list)

def link(a, b):
    if b.key < a.key:
        a, b = b, a
    a.children.append(b)
    return a
```

link принимает деревья одинаковой степени. Степень равна len(node.children).

Python: переносы и meld

Добавление одного дерева

```
def _add_tree(self, tree):
    d = len(tree.children)
    while d in self.roots:
        other = self.roots.pop(d)
        tree = link(tree, other)
        d += 1
    self.roots[d] = tree
```

Слияние двух куч

```
def meld(self, other):
    if self is other:
        raise ValueError("self meld")
    for tree in other.roots.values():
        self._add_tree(tree)
    other.roots.clear()
```

В __init__: self.roots = {}. После meld вторая куча пуста.

Python: извлечение минимума

```
def pop(self):
    if not self.roots:
        raise IndexError("empty heap")
    d = min(self.roots,
        key=lambda d: self.roots[d].key)
    root = self.roots.pop(d)
    for child in root.children:
        self._add_tree(child)
    return root.key
```

Пример использования

```
a = BinomialHeap()
b = BinomialHeap()
a.push(7)
a.push(2)
b.push(1)
a.meld(b)
print(a.pop())  # 1
print(a.pop())  # 2
```

heapq реализует бинарную кучу. Для биномиальной кучи используем собственный класс.